

MP3a: Vector Applications in Interactions

Due Time: Refer to the course web-site

Objective

In this programming assignment, we want to apply concepts in vectors and practice abstractions.

Approach

We will build on the results from MP3 and integrate more advanced functions. Here is [my attempt at implementation](#) for your reference. You are welcome to use my MP3 solution as the base for your MP3a. Be careful, be ready to dedicate time to become familiar with my solution before you can begin working on MP3a.

Form MP3, you have

1. the four walls:
 - LeftWall: P(-17, 0, 12), R(-180, 90, 0), S(34, 34, 1)
 - RightWall: P(17, 0, 12), R(0, 90, 0), S(34, 34, 1)
 - BackWall: P(0, 0, 24), R(0, 0, 0), S(34, 34, 1)
 - Floor: P(0, 0, 12), R(90, 0, 0), S(34, 34, 1)
2. The ability to create/delete **AimLines** and **BigLines**
3. The ability to create/control **TravelingBalls** travel along the AimLines.

For MP3a, you will support: **TheBarrier**. This is a **Plane** with a flattened sphere as its child. The sphere signifies and active region for intersection. Here are the details:

- **Plane:** Again, my Plane is defined based on a Quad. A plane always displays its current Normal direction. This is shown as a **LineSegment** with length of **5** located at the center of the **Plane**.
- **TheBarrier:** S(12, 12, 1), the flatten sphere is a child with S(1, 1, 0.1)
 - Initial settings: P(0, 2, 13), R(0, 75, 0)
- **BarrierControl:** Your user has access to the position and rotation of **TheBarrier**.
- **Front:** The *front* of **TheBarrier** is side where the normal points towards. *You can assume **no** scaling will be applied on TheBarrier plane after your initial settings.*
- **TravelingBall** interaction:
 - A **TravelingBall** casts shadow (a black flattened sphere) and is displayed on **TheBarrier** plane when:
 - The ball is *in front of TheBarrier*, AND
 - Its projected position is *within the active region* as signifies by the flattened sphere.
 - **Indicating line:** For clarity, a thin black line segment (width of 0.02) must connect between a **TravellingBall** and its shadow on the **TheBarrier** plane.
 - **NOTE:** The shadow must respect the orientation (rotations) of **TheBarrier** and always *stay flat* on the plane.
 - A **TravelingBall** will reflect off **TheBarrier** if it collides **TheBarrier** from the front. This is to say, a **TravelingBall** will pass right through **TheBarrier** when approaches from behind.
- **AimLine** interaction:
 - When an **AimLine** intersects **TheBarrier** *within the active region*, the reflection of the line segment that is in front of the **TheBarrier** should be displayed as a separate green line segment.
 - The length of the green line segment should be exactly the same as the segment of **AimLine** that is in front of **TheBarrier**.
 - There is no reflection from the back of **TheBarrier**.
- **Left mouse click:**
 - **On the floor** crates a new instance of **TheBarrier**, all created barriers have same behavior as above
 - Note, with a front- and a back-facing barriers, the traveling ball would bounce between the two

- **On the *active region*** will select and bind a barrier to the **BarrierControl**
 - Make sure you have proper support for two-way binding!
- **On the *region*** outside of *active region* will select and delete the corresponding barrier
- **MP3** LMB support must be retained:
 - Selecting and move **LineEndPt**
 - Left-wall creates AimLine, Back-wall creates BigLine
 - Remove lines when over floor

Lastly for the **BigLine**. You know will support interaction with the **TravelingBall**:

- A **TravelingBall**'s position is projected onto the **BigLine** (show as a white sphere with radius of 0.5), when
 - Its perpendicular distance is less than 10 **AND**
 - Its projected position is within the **BigLine** segment
 - The projected point must be shown at the *surface* of the **BigLine**
 - **Indicating line**: Once again, for clarity, a thin black line segment (width of 0.02) must connect between a **TravellingBall** and its projection on the **BigLine**.
 - A **TravelingBall** reflects off the **BigLine** if a collision should occur.
- **TheBarrier**: reflect off when intersecting from the front in the same way as an AimLine

Hints:

1. In my implementation Sphere/Plane intersection is defined as when the center of sphere (ball) is within 0.1 distant from the plane.
2. If you cannot see the “shadows” on TheBarrier, try hiding the active region sphere, it may be drawn *underneath* the active region sphere. I literally *moved* the shadow-spheres 0.1 units in the plane normal direction to avoid this problem.
3. In my implementation, a TravelingBall has a set of shadow-spheres, projection-spheres, and indicating lines for supporting the projection on the plane and the created BigLines.
4. My implementation sequence after MP3:
 - a. Define **TheBarrier** (in a separate project) with the **BarrierControl**
 - i. Create and manipulate a sphere (pretending it is a TravelingBall)
 - ii. Compute projection for shadow, initially draw a simple sphere as shadow
 - iii. When projection position is properly computed, flatten the sphere
 - iv. Make sure only draw shadow when inside the active region
 - v. Closely examine the functions created for the barrier and extract/define a class (MonoBehavior subclass) to represent a barrier
 - vi. Test that I can create/delete barrier objects (e.g., by hitting on keys)
 - vii. Integrate results project into my MP3
 - b. Take the above project,
 - i. Create and manipulate an AimLine
 - ii. Compute intersection and reflection green line
 - iii. When reflection is working, make sure only reflect from the front and length is only from the front
 - iv. Make sure only triggers reflection when inside the *active region*
 - v. Integrate result to updated MP3
 - c. Define the **BigLine** in a separate project
 - i. Create and manipulate a sphere (to represent the TravelingBall)
 - ii. Compute projection onto the BigLine (cylinder)
 - iii. Compute reflection from the cylinder
 - iv. Make sure only trigger the computation when distance is proper
 - v. Integrate result to updated MP3
 - d. Take step-f, support creation/deletion of new lines
 - i. This is actually rather trivial when everything is working

You should keep projects from each step, such that when bugs occur (not if, but when), you can go back to the previous simpler case, test find your bug and continue. In the very worst case, you can submit these projects showing different portion of your project working.

Warning: A global searched of the **RayCast()** function will be performed, **0% credit will be given to submissions that call RayCast() anywhere other than in supporting mouse click user interaction.** Once again, you **cannot** use Unity Physics (not Kinematic, not Dynamic) for this assignment

Credit Distribution:

- | | | |
|---|------|-------------|
| 1. MP3 functionality does not work anymore | | -80% |
| a. Create/delete AimLine | | |
| b. Traveling ball still under proper control | | |
| c. [no way to test your MP3a] | | |
| 2. TheBarrier proper drawn with transform control | | 20% |
| a. With visible flattened sphere | 5% | |
| b. With proper normal vector | 10% | |
| c. Proper support for GUI position/rotation control | 5% | |
| 3. TheBarrier and TravelingBall interaction | | 20% |
| a. Front: shadow projection in active region | 15% | |
| i. Shadow ball relative size is extra credit (3-pt) | | |
| b. Proper indicating lines connecting to shadow | 10% | |
| c. Reflection: only from front and active region | | 10% |
| d. Reflection: Back: passes through | 5% | |
| e. TheBarrier not interactively manipulatable | -20% | |
| 4. TheBarrier and AimLine interaction | | 20% |
| a. Front reflection when intersect in active region | | |
| i. Direction | 10% | |
| ii. Proper length | 10% | |
| b. No reflection from the back | 10% | |
| c. TheBarrier not interactively manipulatable | -20% | |
| 5. Instances of TheBarrier | | 20% |
| a. Create when clicked on Floor | 10% | |
| b. Select/Delete behavior | 5% | |
| c. Proper two-way bounded to GUI | 5% | |
| d. All created barriers exact same behavior | 10% | |
| e. Behaviors of new instances same as AimLine | 15% | |
| 6. BigLine and TravelingBall | | 15% |
| a. Proper projections | 5% | |
| b. Only project within 10-units | 5% | |
| c. Indicating lines | 5% | |
| d. Proper reflection | 5% | |
| 7. Submission proper zipfile name, Windowed, Resizable, EXE folder | | 5% |

Testing: Make sure to test, for both big and small lines (should be the same function in your code):

- Delete a line when reflection is on
 - Is the green reflection line deleted? (Should be gone)
- Move line end points:
 - Move both in front of the Barrier: Are we seeing the reflection? (should not)

- Move one to the back: see reflection in the front
- Move both back to the front: no reflection
- Move the other to the back: see reflection in the front

Warning: Similar to MP3, using RayCast for selection of AimLine endpoint is fine and expected. However, you will receive an automatic zero for the entire assignment if you use RayCast to solve any other collision challenges. Make sure to talk to me if you see the need to use RayCast in any situation other than mouse-click select. Once again, in your code DO NOT work with Unity Colliders (we are simulating Unity Collider functionality). There is a relatively small amount of code to be written, BUT, this is tricky. Start early! Let me know your questions as soon as you encounter any! **Remember**, Unity Physics engine, i.e., Kinematic, is completely off limit for this entire class, except in your final project. If you have any kinds of OnTriggerAnything() function, something is wrong!

Extra Credit: Some ideas, DO NOT work on the following until you are done with the base assignment. In all cases, you MUST support the default of just showing one. Otherwise, it can be impossible to test your submission.

In this MP, you can think of extra credits (according to your own interests) in (at least) three separate directions:

1. **Art/Design:** Are you interested in scene composition? Do you want to see MP3 in a more interesting setting (rather than the very boring set up)? Or,
2. **Math:** Are you interested in practicing (or demonstrating) your knowledge on math, then, maybe you can include a sphere or something else and project traveling balls onto these different primitives? And/Or reflecting off these primitives.